Exercice 5.1 : Méthode générique

Ecrire une méthode statique `shuffle` qui reçoit un tableau (de n'importe quel type) et qui le mélange à l'aide de l'algorithme suivant : pour chaque élément du tableau, l'échanger avec lui même ou avec un autre élément choisi au hasard plus loin dans le tableau.

L'exemple suivant illustre l'usage de la classe `java.util.Random` pour afficher 5 nombres pseudo-aléatoires dans l'intervalle `[0..100[`

```
Random generator = new Random();
for(int k = 0; k < 5; ++k)
  System.out.println(generator.nextInt(100));
```

Exercice 5.2 : Interface Comparable<T>

Faites en sorte que vos classes `Date` et `Person` implémentent respectivement l'interface `Comparable<Date>` et `Comparable<Person>`.

Exercice 5.3 : Liste et Collections.sort()

Créez une liste de `Date` contenant quelques dates.

Triez-là ensuite à l'aide de `Collections.sort()`.

Faites de même avec une liste de `Person`.

Exercice 5.4 : Stack et Queue (voir le programme d'exemple QueueAndStack)

Les *queues* sont des collections FIFO. Elles exposent (entre autres) les méthodes suivantes :

- `boolean offer(E e)` : Ajoute, si possible, `e` dans cette *queue*. Renvoie `true` si `e` a été ajouté.
- `E poll()` : Supprime et renvoie le 1er élément de cette *queue* ou renvoie `null` si cette *queue* est vide.
- `E peek()` : Renvoie, sans l'enlever, le 1er élément de cette *queue* ou renvoie `null` si cette *queue* est vide.
- `boolean isEmpty()` : Renvoie `true` ssi cette *queue* est vide
- `int size()` : Renvoie le nombre d'éléments dans cette *queue*

Implémentez une classe `MyQueue<E>` à l'aide d'une `List<E>`. Faites fonctionner le projet IntelliJ fourni (`QueueAndStack`) et les tests (commentez momentanément les Stacks).

Les *stacks* sont des collections LIFO. Faites de même pour une classe `MyStack<E>`. Consultez l'aide pour reprendre les spécifications des méthodes `push`, `pop` et `peek` de la classe `Stack<E>` de la librairie.

Exercice 5.5 : Sets

Les *sets* (`Set<E>`) sont des collections qui ne peuvent pas contenir de doublons. Pour cela, ils se basent éventuellement sur les méthodes `equals` et `compareTo` (c'est le cas de `TreeSet`). Ils se créent de la façon suivante :

```
Set<Date> s = new TreeSet<>();
```

Vérifiez leur fonctionnement en créant un *set* de dates et en essayant d'y ajouter des dates identiques.